Research Topic: Accuracy Evaluation

Main Focus: Measuring the proportion of correct predictions.

Group 2: DIN Reaksa, Ngen Tina, KHAY LINA, Mon Sreylin,
Yong Lyhor, PHAL MENGHAK, KEM Sreyoun

Introduction: What is Accuracy?

- It measures the proportion of correct predictions
- It's great for strict classification tasks
- It's mathematically simple

It answers the simple question:
"*Out of all the answers the LLM gave, how many were perfectly correct?*"

Theory

Accuracy tells us how many answers the AI gets right compared with the total number of questions.

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{All predictions}}$$

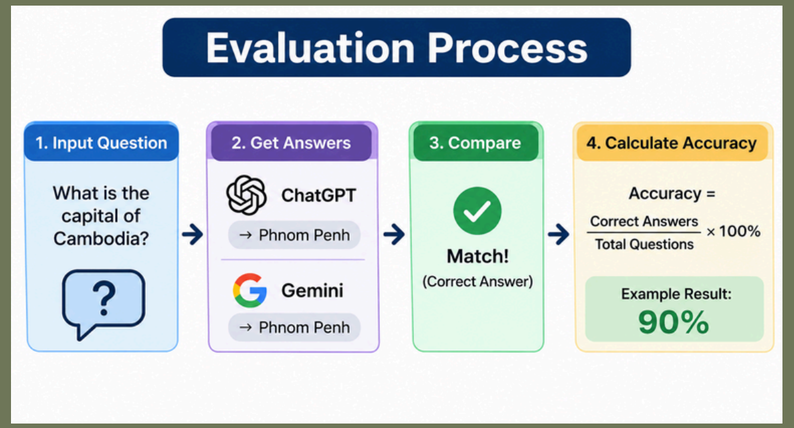
Mathematical formula

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- TP = True Positive
- TN = True Negative
- FP = False Positive
- FN = False Negative

Example:



Suppose an LLM answers 100 questions:

- 90 answers are correct.
- 10 answers are incorrect.

$$Accuracy = \frac{90}{100} \times 100$$

$$Accuracy = 90\%$$

Practical Examples

Santiment Analysis

Input Review	True Label	Model Guess	Result
"Best movie I've seen all year!"	Positive	Positive	Correct
"Boring plot and terrible acting."	Negative	Negative	Correct
"I fell asleep after ten minutes."	Negative	Negative	Correct
"Not bad, actually pretty fun."	Positive	Negative	Wrong

- **Score: 3 / 4 = 75% Accuracy** (Beats the 50% random baseline)

Mulltiple choise

Question Snippet	True Answer	Model Guess	Result
"Capital of France?"	B (Paris)	B (Paris)	Correct
"Chemical symbol for gold?"	A (Au)	A (Au)	Correct
"Author of 1984?"	D (Orwell)	C (Huxley)	Wrong
"Largest planet in solar system?"	C (Jupiter)	C (Jupiter)	Correct

- **Score: 3 / 4 = 75% Accuracy** (Beats the 25% random baseline)

Python Implementation

```python
def calculate_accuracy(predictions, reference_answers):
    correct = 0
    total = len(reference_answers)

    for i in range(total):
        if predictions[i] == reference_answers[i]:
            correct += 1

    return correct / total

# Example Data
refs  = ["Positive", "Negative", "Positive"]
preds = ["Positive", "Positive", "Positive"]
print(f"Accuracy: {calculate_accuracy(preds, refs) * 100}%")
# Output: 66.6%
```

```python
from sklearn.metrics import accuracy_score

# Example Data
refs  = ["Positive", "Negative", "Positive"]
preds = ["Positive", "Positive", "Positive"]
print(f"Accuracy: {accuracy_score(y_true, y_pred) * 100}%")
# Output: 66.6%
```

Experiment

Review	Actual Answer	Gemini 1.5 Prediction	Llama 3 Prediction
1. "Loved it!"	Positive	Positive (Correct)	Positive (Correct)
2. "Terrible acting."	Negative	Negative (Correct)	Negative (Correct)
3. "It was okay."	Positive	Positive (Correct)	Negative (Incorrect)
4. "A masterpiece."	Positive	Positive (Correct)	Positive (Correct)
5. "Boring plot."	Negative	Positive (Incorrect)	Negative (Correct)
Final Score:		**Gemini = 80% (4/5)**	**Llama 3 = 80% (4/5).**

Advantages & Limitations

Strengths:

- Very intuitive and easy to understand.
- Simple mathematical formula to implement.

Limitations:

- Can be highly misleading with "imbalanced datasets".
- Does not tell you what type of mistakes the model is making (false positives vs. false negatives).

Conclusion

- **When to use:** Use when the dataset is "balanced" (e.g., an equal number of positive and negative examples).
- **Why to use:** It provides a fast, high-level snapshot of overall correctness.
- **Best Applications:** Binary classification (Yes/No), multiple-choice question answering, and balanced text classification.

Any Questions?